

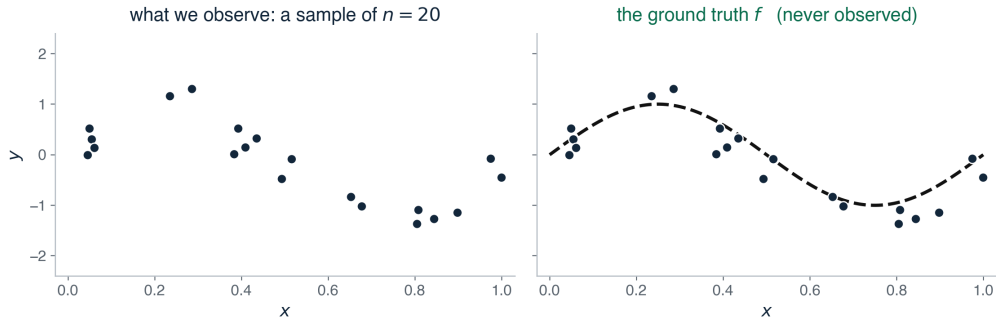
Bias–Variance Trade-off

MATH 4432 Statistical Machine Learning · Tutorial T02

Department of Mathematics, HKUST

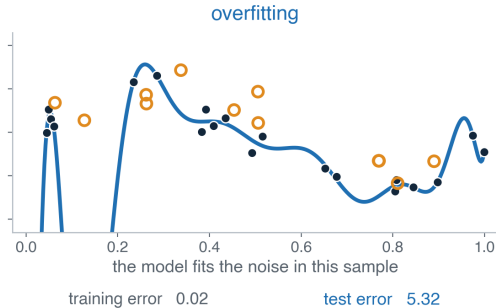
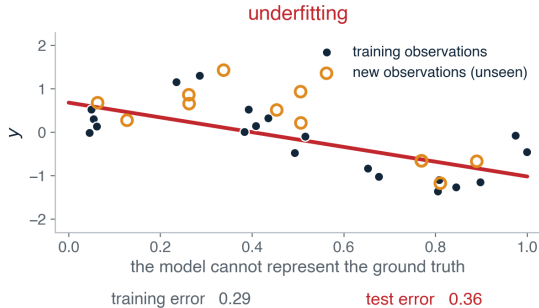
Fall 2026

What we observe, and what we want



- we observe a finite **sample**; we want the **ground truth f**
- we are evaluated on observations that are *not* in the sample

Underfitting and overfitting



a small training error is not evidence of a good model

$$Y = f(X) + \varepsilon, \quad \mathbb{E}[\varepsilon] = 0, \quad \text{Var}(\varepsilon) = \sigma^2$$

- f the **ground truth**. Unknown, fixed.
- ε **irreducible noise**, mean 0, variance σ^2
- $D = \{(x_n, y_n)\}_{n=1}^N$ the **training sample**, drawn at random $\Rightarrow$ the fitted model $\hat{f}(\cdot; D)$ is random too

Which error?

$$\text{Err}(x_0) = \mathbb{E}_{D, \varepsilon} \left[(Y_0 - \hat{f}(x_0; D))^2 \mid X = x_0 \right]$$

Training error

measured on the training sample
always $\downarrow$ with complexity

Test error (generalisation error)

a new observation from the same
distribution
the quantity we want to minimise

Bias–variance decomposition

$$\mathbb{E}[(Y_0 - \hat{f}(x_0))^2] = \underbrace{(f(x_0) - \mathbb{E}[\hat{f}(x_0)])^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)])^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{noise}}$$

Bias²

systematic error
reducible

Variance

sensitivity to the sample
reducible

σ^2

irreducible noise
a lower bound

$$\begin{aligned}\mathbb{E}[(Y_0 - \hat{f})^2] &= \mathbb{E}[(f + \varepsilon - \hat{f})^2] \\ &= \underbrace{\mathbb{E}[\varepsilon^2]}_{\sigma^2} + \mathbb{E}[(f - \hat{f})^2] + \underbrace{2 \mathbb{E}[\varepsilon] \mathbb{E}[f - \hat{f}]}_{=0}\end{aligned}$$

$$\begin{aligned}\mathbb{E}[(f - \hat{f})^2] &= \mathbb{E}\left[\left(\underbrace{f - \mathbb{E}\hat{f}}_{\text{constant}} + \underbrace{\mathbb{E}\hat{f} - \hat{f}}_{\text{mean 0}}\right)^2\right] \\ &= (f - \mathbb{E}\hat{f})^2 + \text{Var}(\hat{f})\end{aligned}$$

Bias and variance

$$\mathbb{E}[\hat{f}(x_0)] - f(x_0)$$

average deviation of $\hat{f}$ from the ground truth

a property of the model class

more data does *not* reduce it

high bias $\Rightarrow$ underfitting

$$\mathbb{E}[(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)])^2]$$

how much $\hat{f}$ changes when the sample is redrawn

sensitivity to noise in the sample

more data *does* reduce it

high variance $\Rightarrow$ overfitting

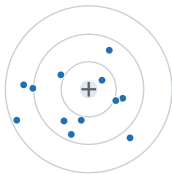
Twelve training sets, twelve predictions

low bias
low variance



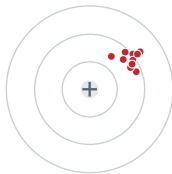
ideal

low bias
high variance



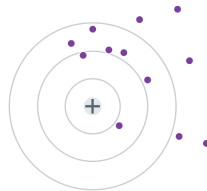
overfitting

high bias
low variance



underfitting

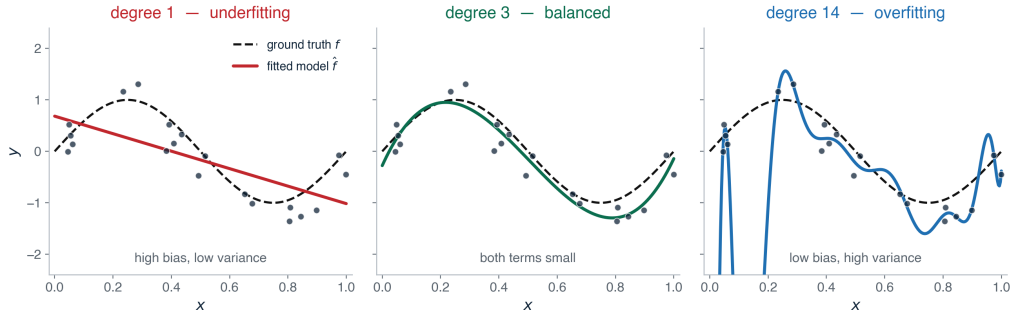
high bias
high variance



worst of both

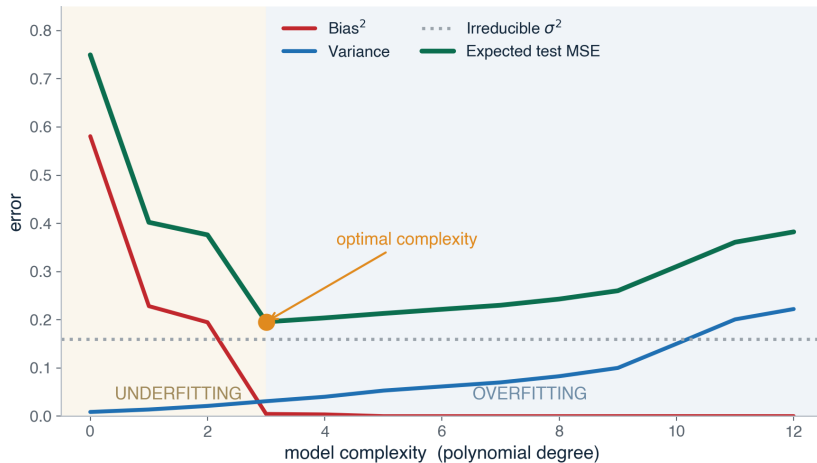
+ = ground truth $f(x_0)$ · each point = the prediction from one training sample

Same data, three models

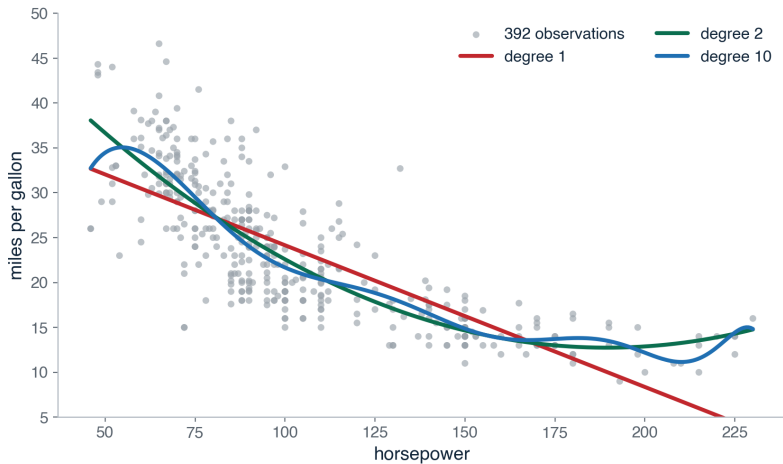


$n = 20$ observations from $y = \sin(2\pi x) + \varepsilon$

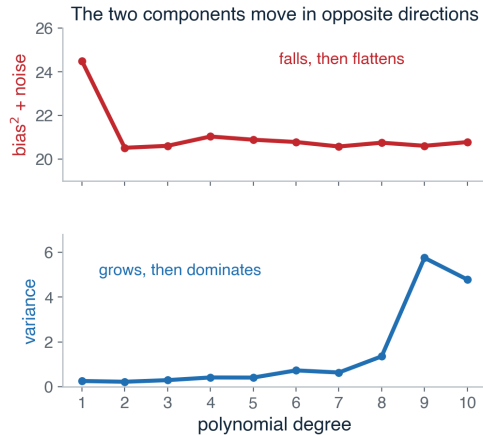
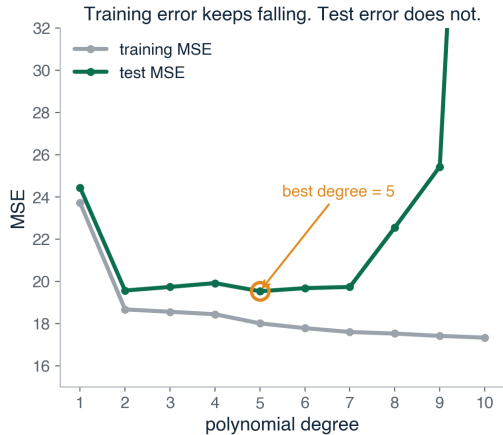
The trade-off



392 cars: mpg vs horsepower



300 random 50/50 splits



f is never observed, so bias^2 and σ^2 cannot be separated

- bias^2 and σ^2 are not separately identifiable
- estimate the generalisation error instead — hold-out, k -fold CV, bootstrap
- take the minimum of the curve; prefer the simplest model within noise of it
- regularise to move along the trade-off

Monte-Carlo estimates

Fix the design $x_1, \dots, x_N$ and the ground truth f ; draw L independent samples.

$$\bar{f}(x) = \frac{1}{L} \sum_{\ell=1}^L \hat{f}^{\ell}(x)$$

$$\text{Bias}^2 = \frac{1}{M} \sum_{m=1}^M (\bar{f}(x_m) - f(x_m))^2$$

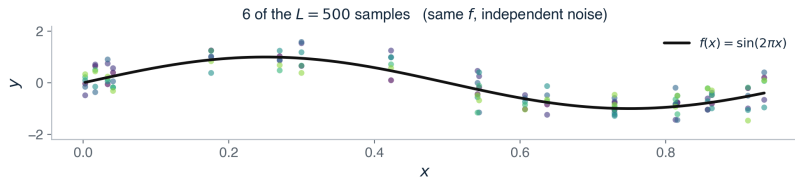
$$\text{Var} = \frac{1}{M} \sum_{m=1}^M \frac{1}{L} \sum_{\ell=1}^L (\hat{f}^{\ell}(x_m) - \bar{f}(x_m))^2$$

1 · simulate L training samples

```
import numpy as np

rng = np.random.default_rng(0)
f = lambda x: np.sin(2 * np.pi * x)
N, L, sigma = 20, 500, 0.4

x_train = np.sort(rng.uniform(0, 1, N))          # design, fixed across samples
Y = f(x_train) + rng.normal(0, sigma, (L, N))    # 500 x 20, one row per sample
```



2 · fit the model to every sample

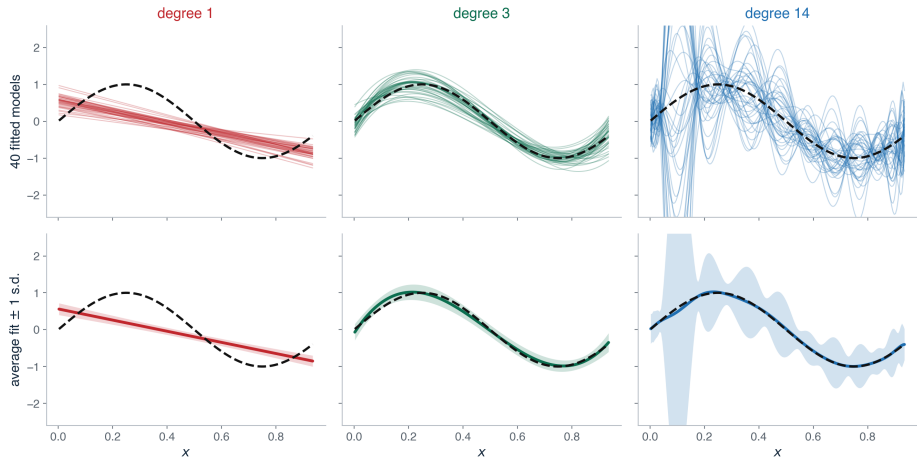
```
def fit_predict(x, y, degree, x_new):  
    p = np.polynomial.Polynomial.fit(x, y, degree)    # rescales x: stable  
    return p(x_new)  
  
x_grid = np.linspace(0, 1, 200)  
  
P = np.vstack([fit_predict(x_train, Y[l], degree, x_grid)  
               for l in range(L)])                  # P is 500 x 200
```

- one row of P = one fitted model
- **bias**: column mean vs the ground truth **variance**: column spread

40 of the $L = 500$ fitted models

spread across samples = VARIANCE

gap between the ground truth and the average fit = BIAS

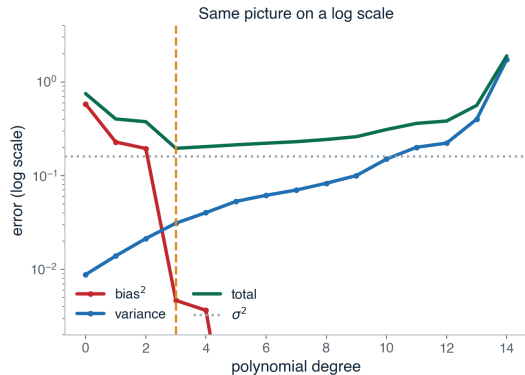
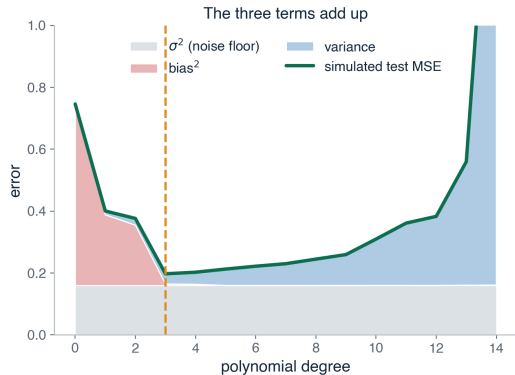


3 · the decomposition

```
f_bar = P.mean(axis=0) # E[f_hat](x)
bias2 = np.mean((f_bar - f(x_grid)) ** 2)
var    = np.mean(P.var(axis=0))
total  = bias2 + var + sigma ** 2
```

degree		bias ²	variance	σ^2	total
1	underfit	0.228	0.014	0.160	0.402
3	best	0.005	0.031	0.160	0.196
14	overfit	0.001	1.734	0.160	1.895

Sweeping the degree



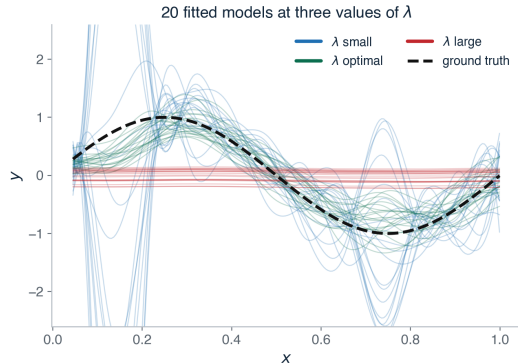
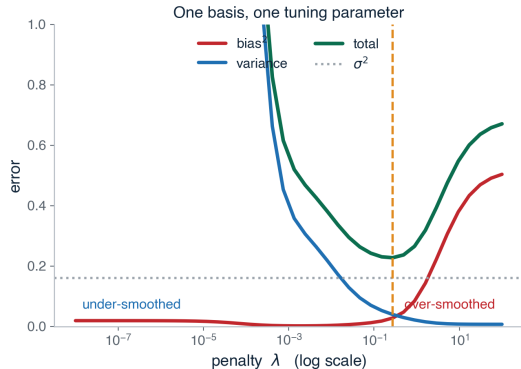
4 · a continuous tuning parameter

$$\hat{\beta}_{\lambda} = \arg \min_{\beta} \sum_{i=1}^N \left(y_i - \sum_j \beta_j B_j(x_i) \right)^2 + \lambda \|\beta\|_2^2$$

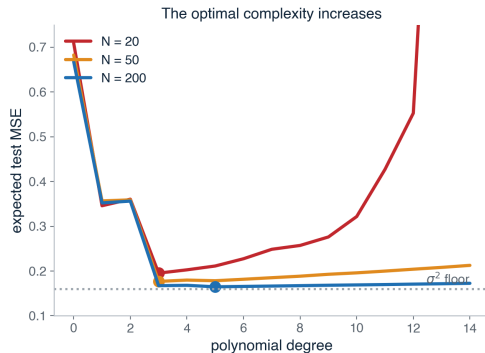
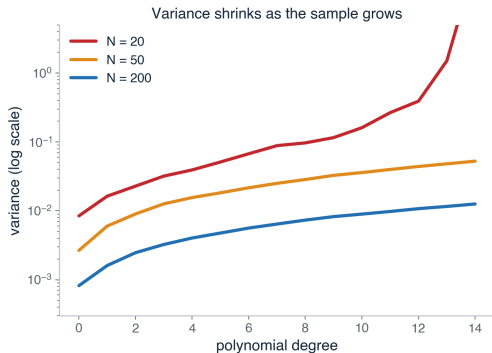
```
from sklearn.preprocessing import SplineTransformer
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline

model = make_pipeline(SplineTransformer(n_knots=12, degree=3), # basis: fixed
                      Ridge(alpha=lam))                      # lam: the tuning parameter
model.fit(x_train[:, None], y)
```

One basis, one tuning parameter



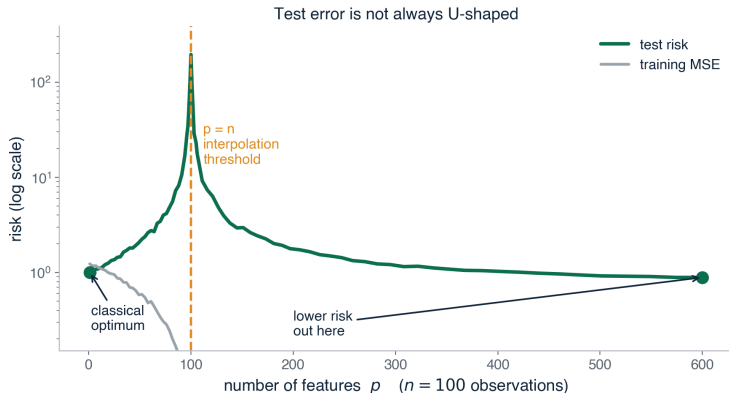
More data $\Rightarrow$ more complexity



$$\text{Var}(\hat{f}) \approx \sigma^2 \frac{p}{N}$$

- generalisation error = $\text{bias}^2 + \text{variance} + \sigma^2$
- flexibility trades one for the other
- zero bias is not the goal; the minimum of the sum is
- training error does not estimate generalisation error
- more data shifts the optimum toward more flexible models

Bonus: double descent



the minimum-norm solution is implicitly regularised, so the variance term falls again
past $p = n$

References

- ISL §2.1–2.2 · ESL §7.3
- Belkin et al. (2019), PNAS — double descent
- `T02_BiasVariance.ipynb` — every experiment on these slides
- `slides/make_figures.py` — every figure on these slides